

Assignment-Module-48

Important Api and annotation

```
Q.1    import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class DateTimeExample {

    // Main method to execute the program
    public static void main(String[] args) {
        // Create an instance of the class
        CurrentDateTime displayDateTime = new CurrentDateTime();
        // Call the method to display the current date and time
        displayDateTime.showDateTime();
    }
}

// Class to handle the display of date and time
class CurrentDateTime {

    // Annotated method to override behavior (if applicable in inheritance)
    @Override
    public String toString() {
        return "This class displays the current date and time.";
    }

    // Method to display the current date and time
    public void showDateTime() {
        // Get the current date and time
        LocalDateTime currentDateTime = LocalDateTime.now();

        // Format the date and time
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

        // Display the formatted date and time
        System.out.println("Current Date and Time: " + currentDateTime.format(formatter));
    }
}
```

```
Q.2 import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
```

```

public class DateToStringExample {

    public static void main(String[] args) {
        // Create a LocalDate object representing today's date
        LocalDate currentDate = LocalDate.now();

        // Define the desired format for the date
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("MM/dd/yyyy");

        // Convert the date to a string in the specified format
        String formattedDate = currentDate.format(formatter);

        // Display the formatted date
        System.out.println("Formatted Date: " + formattedDate);
    }
}

```

Q.3 The main difference between Collections and Streams in Java lies in their purpose and how they handle data. Here's a detailed comparison along with examples:

Aspect	Collections	Streams
Definition	A Collection is a data structure that stores and organizes objects in memory (e.g., List , Set , etc.).	A Stream is a sequence of elements that supports functional-style operations for processing data.
Storage	Stores data in memory (elements persist in the collection).	Does not store data; processes data on demand (data is ephemeral).
Nature	Eager: All elements are available immediately for iteration.	Lazy: Elements are processed only when needed.
Data Processing	Focuses on data storage and retrieval.	Focuses on data processing, transformations, and computations.
Type of Operations	Includes adding, removing, or updating elements (mutability).	Performs functional operations like map , filter , reduce (immutable and declarative).

Iteration	Uses external iteration (<code>for</code> loop, <code>Iterator</code> , etc.).	Uses internal iteration, enabling parallelism.
Parallelism	Parallelism needs to be implemented manually.	Built-in support for parallelism using <code>.parallelStream()</code> .
Mutability	Collections are mutable by default (modifiable elements).	Streams are immutable; once consumed, they cannot be reused.
Use Case	Suitable for storing and managing data.	Suitable for processing data using functional programming.

Q.4 We can use enum to define a group of named constants.

Enums are used to represent a collection of related constants that have a common purpose. Each constant in an enum is an instance of the enum type, and they are typically defined as public static final fields.

Here's an example of how to define an enum in Java:

```
class EnumDemo{
public enum DayOfWeek {
MONDAY,
TUESDAY,
WEDNESDAY,
THURSDAY,
FRIDAY,
SATURDAY,
SUNDAY
}
public static void main(String args[]){
for(DayOfWeek d:DayOfWeek.values())
System.out.println(d);
}
```

```
}
```

Here we define an enum called "DayOfWeek" that represents the days of the week. The enum has seven constants, each representing a day of the week. The constants are defined in all uppercase letters by convention.

Q.5 built-in annotations in Java:

@Override

@Deprecated

@SuppressWarnings

@FunctionalInterface

@Retention

@Target

@Documented

@Inherited

These built-in annotations in Java are used to provide additional information to the Java compiler and other tools. They help improve code readability, maintainability, and safety by enforcing specific rules and behaviours in Java code.